ML Lab — Exam Snippet & Formula Reference

Key Snippets | Formulas | Viva Q&A | Labs 1–10

Lab 1 — Loading Datasets & Train-Test Split

Key Snippet

Load Iris/Digits/Diabetes → DataFrame → split 70:30

```
from sklearn import datasets
import pandas as pd
from sklearn.model_selection import train_test_split

iris = datasets.load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = pd.Series(iris.target)
df['target_names'] = df['target'].apply(lambda y: iris.target_names[y])

print(df.head(5))    # first 5 rows
print(df.sample(5))  # 5 random rows

df_train, df_test = train_test_split(df, test_size=0.3)
print('Train:', df_train.shape[0], '  Test:', df_test.shape[0])
```

Lab 2 — Train / Validation / Test Split

Key Snippet

▶️ **Two consecutive splits to create train, val, test**

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    df, label, test_size=0.3, shuffle=True)

X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size=0.3, shuffle=True)

print("Train:", X_train.shape[0])
print("Val:  ", X_val.shape[0])
print("Test: ", X_test.shape[0])
```

Lab 3 — K-Fold Cross Validation

Formulas

Fold size

```
fold_size = N // K
```

Mean CV Accuracy

```
Mean Accuracy = (1/K) × Σ[i=1..K]  accuracy_i
```

💡 *Each fold: train on K-1 parts, test on the remaining 1 part.*

Key Snippet

▶️ **Manual K-Fold + Decision Tree evaluation**

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

def kfold_indices(data, k):
    fold_size = len(data) // k
    indices = np.arange(len(data))
    folds = []
    for i in range(k):
        test_idx  = indices[i*fold_size : (i+1)*fold_size]
        train_idx = np.concatenate([indices[:i*fold_size],
                                    indices[(i+1)*fold_size:]])
        folds.append((train_idx, test_idx))
    return folds

model, scores = DecisionTreeClassifier(), []
for train_idx, test_idx in kfold_indices(X, k=10):
    model.fit(X[train_idx], y[train_idx])
    scores.append(accuracy_score(y[test_idx], model.predict(X[test_idx])))

print("Mean Accuracy:", np.mean(scores))
```

Lab 4 — Confusion Matrix, ROC Curve & Upsampling

Formulas — Classification Metrics

Accuracy

```
Accuracy  =  (TP + TN) / (TP + TN + FP + FN)
```

Precision

```
Precision =  TP / (TP + FP)
```

Recall (Sensitivity / TPR)

```
Recall    =  TP / (TP + FN)
```

F1 Score

```
F1        =  2 × (Precision × Recall) / (Precision + Recall)
```

False Positive Rate (FPR) — x-axis of ROC

```
FPR       =  FP / (FP + TN)
```

💡 *ROC Curve: TPR (y-axis) vs FPR (x-axis). AUC closer to 1 = better model.*

Confusion Matrix Layout

```
                Predicted Positive   Predicted Negative
Actual Positive       TP                   FN
Actual Negative       FP                   TN
```

Key Snippets

▶️ Confusion Matrix + Report + ROC

```
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
from sklearn import metrics
import matplotlib.pyplot as plt

actual    = [1,0,0,1,0,1,1,1,0,1,0]
predicted = [1,0,0,1,0,0,0,1,0,0,1]

print(confusion_matrix(actual, predicted, labels=[1,0]))
print('Accuracy:', accuracy_score(actual, predicted))
print(classification_report(actual, predicted, labels=[1,0]))

fpr, tpr, _ = metrics.roc_curve(actual, predicted)
plt.plot(fpr, tpr)
plt.xlabel('FPR'); plt.ylabel('TPR')
plt.show()
```

▶️ Upsampling minority class

```
from sklearn.utils import resample
df_majority = df[df.target == 0]
df_minority = df[df.target == 1]
df_minority_up = resample(df_minority, replace=True,
                          n_samples=130, random_state=6)
df_balanced = pd.concat([df_majority, df_minority_up])
```

Lab 5 — Linear Regression via Gradient Descent

Formulas

Hypothesis

$$Y_pred = m \times X + c$$

Mean Squared Error (MSE) Loss

$$L = (1/n) \times \Sigma\ (Y - Y_pred)^2$$

Partial derivative w.r.t. m

$$\partial L/\partial m = (-2/n) \times \Sigma\ X \cdot (Y - Y_pred)$$

Partial derivative w.r.t. c

$$\partial L/\partial c = (-2/n) \times \Sigma\ (Y - Y_pred)$$

Parameter Update Rules

$$m = m - L \times \partial L/\partial m \quad c = c - L \times \partial L/\partial c$$

💡 *L = learning rate (step size). epochs = number of update iterations.*

Key Snippet

▶️ **Gradient descent loop**

```
m, c  = 0, 5000
L     = 0.0001   # learning rate
n     = float(len(X))

for i in range(1000):
    Y_pred = m*X + c
    D_m = (-2/n) * sum(X * (Y - Y_pred))   # dL/dm
    D_c = (-2/n) * sum(Y - Y_pred)          # dL/dc
    m   = m - L * D_m
    c   = c - L * D_c

print(f"m = {m:.4f}, c = {c:.4f}")
```

Lab 6 — Bias-Variance Tradeoff

Formulas

Bias-Variance Decomposition

$$MSE = Bias^2 + Variance + Irreducible\ Noise$$

Bias (systematic error)

$$Bias = E[Y_pred] - Y_true$$

Variance (sensitivity to training data)

$$Variance = E[(Y_pred - E[Y_pred])^2]$$

Mean Squared Error

$$MSE = (1/n) \times \Sigma\ (Y_true - Y_pred)^2$$

```
Model Complexity vs Error:
  Low complexity  →  High Bias,    Low Variance  (underfitting)
  High complexity →  Low Bias,     High Variance (overfitting)
  Optimal         →  Balanced Bias & Variance     (best generalization)
```

Key Snippet

Bias-Variance decomposition using mlxtend

```
from mlxtend.evaluate import bias_variance_decomp
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# High Bias model
linear = LinearRegression()
linear.fit(X_train, y_train)

# High Variance model
poly = make_pipeline(PolynomialFeatures(degree=15), LinearRegression())
poly.fit(X_train, y_train)

for model, name in [(linear,"Linear"), (poly,"Poly degree-15")]:
    mse, bias, var = bias_variance_decomp(
        model, X_train, y_train, X_test, y_test,
        loss='mse', num_rounds=200, random_seed=1)
    print(f"{name}: MSE={mse:.3f}  Bias={bias:.3f}  Var={var:.3f}")
```

Lab 7 — Decision Tree

Formulas

Gini Impurity (classification criterion)

$$Gini(S) = 1 - \Sigma[k] \; p_k^2$$

Entropy

$$H(S) = -\Sigma[k] \; p_k \times \log_2(p_k)$$

Information Gain

$$IG(S,A) = H(S) - \Sigma[v] \; (|S_v|/|S|) \times H(S_v)$$

MSE criterion (regression trees)

$$MSE = (1/n) \times \Sigma \; (y_i - \bar{y})^2 \qquad \bar{y} = \text{mean of leaf node}$$

MAE criterion (regression trees)

$$MAE = (1/n) \times \Sigma \; |y_i - \text{median}| \qquad \text{median of leaf node}$$

💡 *p_k = proportion of class k in node S. Splitting maximizes Information Gain / minimizes Gini.*

Key Snippets

▶️ Classifier: fit → predict → visualize

```
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn import tree
import matplotlib.pyplot as plt

clf = DecisionTreeClassifier()        # add max_depth=5 to limit size
clf.fit(x_train, y_train)
y_pred = clf.predict(x_test)

print(export_text(clf))               # text tree

fig = plt.figure(figsize=(25, 20))
tree.plot_tree(clf,
    feature_names=iris.feature_names,
    class_names=iris.target_names, filled=True)
plt.show()
```

▶️ Accuracy evaluation

```
from sklearn.metrics import accuracy_score
print('Accuracy:', accuracy_score(y_test, y_pred))
```

▶️ Regressor with two criteria

```
from sklearn.tree import DecisionTreeRegressor
for criterion in ['squared_error', 'absolute_error']:
    rgrsr = DecisionTreeRegressor(criterion=criterion)
    rgrsr.fit(x_train, y_train)
    print(export_text(rgrsr, feature_names=['Gender']))
```

Lab 8 — Local Outlier Factor (LOF) & KNN

Formulas — LOF

Reachability Distance

$$\text{reach-dist}_k(A,B) = \max(\text{k-dist}(B), \text{dist}(A,B))$$

Local Reachability Density

$$\text{lrd}_k(A) = k \,/\, \Sigma[B \in N_k(A)] \ \text{reach-dist}_k(A,B)$$

LOF Score

$$\text{LOF}_k(A) = (1/k) \times \Sigma[B \in N_k(A)] \ \text{lrd}(B) \,/\, \text{lrd}(A)$$

💡 *LOF ≈ 1: normal point. LOF >> 1: outlier (much lower density than neighbours).*

Formulas — KNN

Euclidean Distance (2D)

$$d(p,q) = \sqrt{[(x_p - x_q)^2 + (y_p - y_q)^2]}$$

Euclidean Distance (n-D)

$$d(p,q) = \sqrt{[\Sigma[i=1..n] (p_i - q_i)^2]}$$

💡 *KNN classifies a point by the majority class among its K nearest neighbours.*

Key Snippets

▶️ LOF outlier detection

```
from sklearn.neighbors import LocalOutlierFactor
import numpy as np, matplotlib.pyplot as plt

X_inliers  = np.r_[0.3*np.random.randn(4,2)+2,
                   0.3*np.random.randn(4,2)-2]
X_outliers = np.random.uniform(-4, 4, size=(2,2))
X = np.r_[X_inliers, X_outliers]

clf     = LocalOutlierFactor(n_neighbors=3, contamination=0.1)
y_pred  = clf.fit_predict(X)
scores  = clf.negative_outlier_factor_

radius = (scores.max() - scores) / (scores.max() - scores.min())
plt.scatter(X[:,0], X[:,1], color='k', s=3)
plt.scatter(X[:,0], X[:,1], s=1000*radius,
            edgecolors='r', facecolors='none')
plt.show()
```

▶️ KNN: compare k=1 vs k=5

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

for k in [1, 5]:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    print(f"k={k}: {accuracy_score(y_test, knn.predict(X_test))*100:.2f}%")
```

▶️ Find best K using validation error

```
from sklearn.preprocessing import StandardScaler
import numpy as np

scaler = StandardScaler()
X_tr  = scaler.fit_transform(X_train)
X_v   = scaler.transform(X_val)
X_te  = scaler.transform(X_test)

val_errors = []
for k in range(1, 101):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_tr, y_train)
    val_errors.append(1 - accuracy_score(y_val, knn.predict(X_v)))

best_k = np.argmin(val_errors) + 1
print("Optimal K:", best_k)
```

Lab 9 — Support Vector Machine (SVM)

Formulas

Hyperplane Equation

$$w^T \cdot x + b = 0$$

Margin (distance between support vectors)

$$\text{Margin} = 2 / \|w\|$$

SVM Objective (maximize margin)

$$\text{Minimize} \quad (1/2)\|w\|^2 \qquad \text{subject to} \quad y_i(w^T x_i + b) \geq 1$$

Soft-Margin SVM (with slack ξ, penalty C)

$$\text{Minimize} \quad (1/2)\|w\|^2 + C\Sigma\xi_i \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

RBF Kernel

$$K(x,z) = \exp(-\gamma \times \|x - z\|^2)$$

Polynomial Kernel

$$K(x,z) = (x^T z + r)^d \qquad d = \text{degree}$$

Sigmoid Kernel

$$K(x,z) = \tanh(\alpha\, x^T z + r)$$

```
Effect of C:
  Large C  → small margin, fewer misclassifications (risk: overfitting)
  Small C  → large margin, allows misclassifications (better generalization)
```

Key Snippets

▶️ Linear SVM + support vector plot

```
from sklearn import svm
from sklearn.inspection import DecisionBoundaryDisplay

clf = svm.SVC(kernel="linear", C=9000)
clf.fit(X, y)

ax = plt.gca()
DecisionBoundaryDisplay.from_estimator(clf, X,
    plot_method="contour", colors="k",
    levels=[-1, 0, 1], linestyles=["--","-","--"], ax=ax)

ax.scatter(clf.support_vectors_[:,0],
           clf.support_vectors_[:,1],
           s=100, facecolors='none', edgecolors='k')
plt.show()
```

▶️ Non-linear SVM on XOR data (NuSVC)

```
from sklearn import svm
import numpy as np, matplotlib.pyplot as plt
```

```
X = np.random.randn(300, 2)
Y = np.logical_xor(X[:,0] > 0, X[:,1] > 0)

clf = svm.NuSVC(gamma="auto")
clf.fit(X, Y)

xx, yy = np.meshgrid(np.linspace(-3,3,500), np.linspace(-3,3,500))
Z = clf.decision_function(np.c_[xx.ravel(),yy.ravel()]).reshape(xx.shape)
plt.imshow(Z, extent=(-3,3,-3,3), origin="lower", cmap=plt.cm.PuOr_r)
plt.contour(xx, yy, Z, levels=[0], linewidths=2, linestyles="dashed")
plt.show()
```

Lab 10 — K-Means Clustering & PCA

Formulas — K-Means

Objective (Minimize Inertia / WCSS)

$$J = \Sigma[k=1..K] \; \Sigma[x \in C_k] \; \|x - \mu_k\|^2$$

Centroid Update

$$\mu_k = (1/|C_k|) \times \Sigma[x \in C_k] \; x$$

💡 *Inertia = kmeans.inertia_. Elbow point = K where inertia drop slows significantly.*

Formulas — PCA

Covariance Matrix

$$\Sigma = (1/(n-1)) \times X^T X \qquad \text{(on zero-mean data)}$$

Eigenvalue equation

$$\Sigma v = \lambda v \qquad (\lambda = \text{eigenvalue}, \; v = \text{eigenvector / principal component})$$

Explained Variance Ratio for component i

$$EVR_i = \lambda_i / \Sigma[j] \; \lambda_j$$

Projection onto k components

$$X_{reduced} = X \times W_k \qquad W_k = \text{top-k eigenvectors}$$

💡 *StandardScaler before PCA: z = (x - mean) / std. Then pca.fit_transform(X_train) only.*

Key Snippets

▶️ K-Means + Elbow method

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

data = list(zip(x, y))
inertias = []
for i in range(1, 11):
    km = KMeans(n_clusters=i, random_state=42)
    km.fit(data)
    inertias.append(km.inertia_)

plt.plot(range(1,11), inertias, marker='o')
plt.title('Elbow Method'); plt.show()

km = KMeans(n_clusters=2, random_state=42)
km.fit(data)
plt.scatter(x, y, c=km.labels_); plt.show()
```

▶️ PCA + Logistic Regression pipeline

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score

# 1. Scale
```

```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test  = sc.transform(X_test)          # transform only!

# 2. PCA
pca = PCA(n_components=2)
X_train = pca.fit_transform(X_train)
X_test  = pca.transform(X_test)         # transform only!
print("Explained Variance:", pca.explained_variance_ratio_)

# 3. Classify
clf = LogisticRegression(random_state=0)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred) * 100, "%")
```

Formula Quick-Reference Sheet

All key formulas in one place for last-minute revision.

[Lab 5] Hypothesis
$Y = mX + c$

[Lab 5] MSE Loss
$L = (1/n)\Sigma(Y - Y_pred)^2$

[Lab 5] Gradient Descent Update
$m = m - \alpha\cdot(-2/n)\Sigma X(Y-Y_pred)$ $c = c - \alpha\cdot(-2/n)\Sigma(Y-Y_pred)$

[Lab 4] Accuracy
$(TP + TN) / (TP + TN + FP + FN)$

[Lab 4] Precision
$TP / (TP + FP)$

[Lab 4] Recall
$TP / (TP + FN)$

[Lab 4] F1 Score
$2 \times (P \times R) / (P + R)$

[Lab 4] FPR
$FP / (FP + TN)$

[Lab 6] Bias-Variance
$MSE = Bias^2 + Variance + Noise$

[Lab 7] Gini Impurity
$1 - \Sigma\ p_k^2$

[Lab 7] Entropy
$-\Sigma\ p_k\ \log_2(p_k)$

[Lab 7] Info Gain
$H(S) - \Sigma(|S_v|/|S|)\cdot H(S_v)$

[Lab 8] Euclidean Distance
$d = \sqrt{\Sigma(p_i - q_i)^2}$

[Lab 8] LOF Score
$LOF(A) = (1/k)\Sigma\ lrd(B)/lrd(A)$

[Lab 9] SVM Margin
$2 / \|w\|$

[Lab 9] SVM Objective
$Minimize\ (1/2)\|w\|^2 + C\Sigma\xi_i$

[Lab 9] RBF Kernel
$K(x,z) = \exp(-\gamma\|x-z\|^2)$

[Lab 10] K-Means Inertia
$J = \Sigma_k\ \Sigma_\{x\in C_k\}\ \|x - \mu_k\|^2$

[Lab 10] Centroid Update

```
μ_k = (1/|C_k|) Σ x
```

[Lab 10] PCA Explained Variance

```
EVR_i = λ_i / Σλ_j
```

[Lab 10] StandardScaler

```
z = (x - mean) / std
```

Viva Questions & Answers

Lab 1 & 2 — Datasets & Splitting

Q: What is train_test_split and why do we use it?

A: It randomly splits data into training and testing subsets so the model is evaluated on data it hasn't seen during training. test_size=0.3 means 30% goes to test.

Q: Difference between df.head(5) and df.sample(5)?

A: head(5) gives the first 5 rows in order. sample(5) picks 5 rows randomly from anywhere in the dataset.

Q: Why do we need a validation set?

A: To tune hyperparameters during training without touching the test set. The test set is kept completely unseen until final evaluation to give an unbiased estimate.

Lab 3 — K-Fold CV

Q: What is K-Fold Cross Validation?

A: The dataset is split into K equal parts (folds). The model trains on K-1 folds and tests on the remaining fold. This is repeated K times and accuracy is averaged. Gives a more reliable estimate than a single split.

Q: What is the formula for Mean CV Accuracy?

A: Mean Accuracy = $(1/K) \times \Sigma$ accuracy_i (average of all K fold scores)

Lab 4 — Metrics

Q: Define TP, TN, FP, FN.

A: TP: predicted positive, actually positive. TN: predicted negative, actually negative. FP: predicted positive, actually negative (Type I error). FN: predicted negative, actually positive (Type II error).

Q: Write the formula for Precision, Recall, F1.

A: Precision = TP/(TP+FP). Recall = TP/(TP+FN). F1 = 2×(P×R)/(P+R).

Q: What is the ROC curve?

A: A plot of TPR (Recall) on Y-axis vs FPR on X-axis at various thresholds. AUC (Area Under Curve) close to 1 means excellent model; 0.5 means random guessing.

Q: Why is upsampling done?

A: To fix class imbalance. Without it the model is biased toward the majority class. resample() with replace=True creates synthetic copies of the minority class.

Lab 5 — Linear Regression / Gradient Descent

Q: What is the hypothesis in linear regression?

A: Y_pred = m×X + c, where m is the slope (weight) and c is the intercept (bias).

Q: What is the loss function and its formula?

A: MSE (Mean Squared Error): $L = (1/n) \times \Sigma(Y - Y_pred)^2$. We minimize this.

Q: Write the gradient descent update equations.

A: $D_m = (-2/n)\Sigma X(Y-Y_pred)$, $m = m - L \cdot D_m$ $D_c = (-2/n)\Sigma(Y-Y_pred)$, $c = c - L \cdot D_c$

Q: What is the effect of the learning rate?

A: Too high: overshoots minimum, may diverge. Too low: very slow convergence. Typical values: 0.001–0.0001.

Lab 6 — Bias-Variance

Q: State the Bias-Variance decomposition formula.

A: MSE = $Bias^2$ + Variance + Irreducible Noise.

Q: What is Bias and Variance individually?

A: Bias: error from wrong assumptions — model too simple (underfitting). Variance: error from sensitivity to training data — model too complex (overfitting).

Q: How does polynomial degree affect bias and variance?

A: Degree 1 (linear): high bias, low variance. Degree 15: low bias, high variance. Optimal degree balances both.

Lab 7 — Decision Tree

Q: What is Gini Impurity? Write the formula.

A: Measures how often a randomly chosen element would be incorrectly classified: $Gini(S) = 1 - \Sigma\, p_k^2$. Pure node: Gini = 0.

Q: What is Entropy? Write the formula.

A: Measures uncertainty in a node: $H(S) = -\Sigma\, p_k \times \log_2(p_k)$. Pure node: H = 0.

Q: What is Information Gain?

A: $IG(S,A) = H(S) - \Sigma(|S_v|/|S|)\times H(S_v)$. The reduction in entropy after splitting on attribute A. The tree splits on the attribute with the highest IG.

Q: Difference between squared_error and absolute_error in regression trees?

A: squared_error: each leaf predicts the mean; minimizes MSE. absolute_error: each leaf predicts the median; minimizes MAE and is more robust to outliers.

Lab 8 — LOF & KNN

Q: What is the Euclidean distance formula?

A: $d(p,q) = \sqrt{[\Sigma(p_i - q_i)^2]}$. In 2D: $\sqrt{[(x_p-x_q)^2 + (y_p-y_q)^2]}$.

Q: How does LOF detect outliers?

A: LOF compares local density of a point to its neighbours. LOF ≈ 1: normal. LOF >> 1: much lower density than neighbours → outlier.

Q: What is negative_outlier_factor_ in LOF?

A: The raw LOF score (negative sign so sklearn convention: more negative = more outlier). The circle radius in the plot is proportional to (max - score)/(max - min).

Q: Why scale features before KNN?

A: KNN uses Euclidean distance. Features with large ranges dominate. StandardScaler (z = (x-mean)/std) puts all features on equal footing.

Lab 9 — SVM

Q: Write the SVM hyperplane and objective formula.

A: Hyperplane: $w^T x + b = 0$. Objective: minimize $(1/2)\|w\|^2 + C\Sigma\xi_i$ subject to $y_i(w^T x_i+b) \geq 1-\xi_i$.

Q: What is the margin in SVM?

A: The distance between the two support vector hyperplanes: Margin = $2/\|w\|$. SVM maximizes this margin.

Q: What is the RBF kernel formula?

A: $K(x,z) = \exp(-\gamma\, \|x-z\|^2)$. It maps data to infinite dimensions to find a separating hyperplane for non-linear data.

Q: Effect of C in SVM?

A: Large C: narrow margin, fewer misclassifications, risk of overfitting. Small C: wider margin, more misclassifications allowed, better generalization.

Q: What are support vectors?

A: Training points closest to the decision boundary. They define the margin. clf.support_vectors_ gives their coordinates.

Lab 10 — K-Means & PCA

Q: Write the K-Means objective (inertia) formula.

A: $J = \Sigma[k=1..K]\ \Sigma[x \in C_k]\ \|x - \mu_k\|^2$. Minimize total within-cluster squared distance.

Q: How is the centroid updated in K-Means?

A: $\mu_k = (1/|C_k|) \times \Sigma[x \in C_k]\ x$ (mean of all points in cluster k).

Q: What is the Elbow Method?

A: Plot inertia vs K. Choose K at the 'elbow' where adding more clusters gives diminishing inertia reduction.

Q: What does PCA do mathematically?

A: Computes eigenvectors of the covariance matrix. Projects data onto the top-k eigenvectors (principal components) that capture the most variance: $X_{reduced} = X \times W_k$.

Q: What is explained_variance_ratio_?

A: $EVR_i = \lambda_i / \Sigma\lambda_j$. Fraction of total variance captured by each principal component. e.g. [0.73, 0.23] means PC1 captures 73%, PC2 captures 23% = 96% total.

Q: Why use pca.transform (not fit_transform) on test data?

A: fit_transform computes new axes from test data → data leakage. transform applies the SAME axes learned from training data, keeping both in the same space.

Q: Write the StandardScaler formula.

A: $z = (x - \mu) / \sigma$ where μ = mean, σ = std of the training set.